KENNESAW STATE
UNIVERSITY

CS 4267
MACHINE LEARNING

PROJECT 2

SUPERVISED LEARNING

Due date: 11/14

INSTRUCTOR

Emin Mary Abraham

Tabitha Washington

1. ABSTRACT

The goal of this project was to develop a basic K-Nearest Neighbor (kNN) classifier for the given dataset. Once the implementation was complete, I needed to test the classifier on the dataset using k values of 1, 3, 5, 7, and 9. This was to be completed without using built-in kNN functions.

In order to complete the project, I implemented a Euclidean distance calculation method, kNN classification method, and several helper methods. The results showed that more neighbors does not necessarily mean better results. The accuracy was highest for 3 and 5 neighbors at 99.5%.

2. METHODOLOGY

The first step of the project was to determine and discuss whether the dataset needed to be normalized or not. The ranges of the values in the dataset indicated that normalization was necessary. Some values were very large while others were very small. When using kNN classification, if one attribute has large values and another has small values, the attribute with large values will dominate the distance comparison. Due to this, it is necessary to normalize the data for this dataset before using kNN classification. There are several different ways to normalize data. I chose to use min max normalization. This decision was due to there being some very large and some very small values in the dataset. Min max normalization normalizes all values to be within the same range. This put all attributes on the same scale and prevented larger valued attributes from dominating the distance measurements.

The next steps handled the data preparation. I used pandas to read the csv. Then I normalized the dataset using MinMaxScaler() from the scikit-learn library. Once I had the normalized dataframe, the data needed to be split. For the data splitting, I sampled the original dataframe to create the training dataframe. Sampling a dataframe is random by default, but I provided a seed to allow reproducibility of the split. Then I created the testing dataframe by dropping the indices in the training dataframe from the original dataframe.

The next step was handling the distance measurements. I created a function that calculates Euclidean distance between the current test sample and each of the training samples. This function returns a list that contains tuples of the training sample to test sample distance measurement and the training sample label.

With the data prepared and the method for the distance calculations complete, I began work on the actual kNN classifier. The kNN classification method accepts a dataframe and the k value as arguments. The classifier tracks the true positive, true negative, false positive, and false negative metrics. It calls a helper method for splitting the data. It then converts the returned training and testing dataframes to numpy arrays. Next, it iterates through each training sample in the test array. It calls the helper methods for calculating the Euclidean distance, finding the closest k number of training samples, and classifying the test sample based on the closest k number of training samples. It then increments the correct metric and appends the predicted class to the results list. Once the test sample iteration is complete, the accuracy is calculated and the confusion matrix is created. The method returns the results list, accuracy, and confusion matrix.

I ran the kNN classification by using the normalized dataframe and specified k value as input. I printed the results list and accuracy, and I displayed the confusion matrix as a dataframe with positive and negative column headers and indices.

2. TEST RESULTS

2.1 Results when k=1

The results from using just one neighbor to perform classification were quite good. The accuracy was 96.7% which is higher than I expected. The results, accuracy printout, and confusion matrix are displayed below.

```
Results: [0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0,
0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1,
0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0,
0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1,
0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1,
0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0,
1, 0, 0, 1, 0, 0, 0]

Accuracy: 0.967391304347826
```

	+	-
+	65	3
-	3	113

2.2 Results when k=3

The results from using just three neighbors to perform classification were a bit better than using one neighbor and very good. The accuracy was 99.5% which is much higher than I expected. I did not anticipate that there would be this significant of a jump from one neighbor to three. The results, accuracy printout, and confusion matrix are displayed below.

```
Results: [0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0,
0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1,
0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0,
0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1,
0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1,
0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0,
1, 0, 0, 1, 0, 0, 0]

Accuracy: 0.9945652173913043
```

	+	-
+	67	1
-	0	116

2.3 Results when k=5

The results from using five neighbors to perform classification were about the same as using three neighbors. The accuracy remained 99.5%. This matches my expectations as I anticipated the accuracy would remain the same or slightly better than with three neighbors. The results, accuracy printout, and confusion matrix are displayed below.

```
Results: [0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0,
0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1,
0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0,
0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1,
0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1,
0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0,
1, 0, 0, 1, 0, 0, 0]

Accuracy: 0.9945652173913043
```

	+	-
+	67	1
-	0	116

2.4 Results when k=7

The results from using seven neighbors to perform classification was slightly worse than using five neighbors. The accuracy lowered to 98.4%. I found this surprising as I expected the accuracy to remain the same as five neighbors or slightly improve. The results, accuracy printout, and confusion matrix are displayed below.

```
Results: [0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0,
0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1,
0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0,
0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1,
0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1,
0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 0,
1, 0, 0, 1, 0, 0, 0]

Accuracy: 0.9836956521739131
```

	+	-
+	65	3
-	0	116

2.5 Results when k=9

The results from using nine neighbors to perform classification was slightly worse than using seven neighbors. The accuracy lowered to 97.8%. After seeing the results from the five to seven neighbors, I was not particularly surprised as this followed the trend between those results. The results, accuracy printout, and confusion matrix are displayed below.

```
Results: [0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0,
0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1,
0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0,
0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1,
0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1,
0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 0,
1, 0, 0, 1, 0, 0, 0]

Accuracy: 0.9782608695652174
```

	+	-
+	64	4
-	0	116

2. DISCUSSION

The tests that I performed for k values of 1, 3, 5, 7, and 9 yielded somewhat surprising results. I originally anticipated that the k=1 results would be quite poor and that test results would gradually improve, however, this was not the case. Instead, k=1 had a higher accuracy than expected, 96.7%. Additionally, while the accuracy did improve from 1 to 3 neighbors, the accuracy plateaued between 3 and 5 neighbors, and it decreased from 5 to 7 and 7 to 9 neighbors. This revealed that the optimal number of neighbors for this particular dataset when using MinMax normalization is either 3 or 5 neighbors.

This project and the experiments with different k values revealed that more neighbors is not always better. The optimal number of neighbors is dependent on the dataset and features used.

My experience working on this project was quite good. The kNN classification was far easier to implement than I expected. The implementation was relatively straightforward. I did run into some minor difficulties, but none of them took particularly long to resolve.

During this project, I noticed several things that I could potentially experiment with if given additional time. Firstly, it could be interesting to experiment with different types of normalization. I chose MinMax because it seemed that it would be the best based on the data. However, it is possible that there are other types of normalization, or different ranges of MinMax, that could work better. Additionally, I would like to consider experimenting with different sets of attributes. It is possible that some attributes are more important than others, and experimentation on subsets of the dataset could reveal the more important attributes.

4. CODES

Note: This code was designed to run in a Python notebook using several different code cells.

4.0 Imports

```
# Imports needed
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
```

4.1 Functions for distance calculations

```
# Calculate the Euclidean distance between the training samples and testing
samples
def calculate_distances(train_arr, test_sample):
    distances = []

    # Exclude the label from the attributes
    test_attributes = test_sample[:-1]

    for train_sample in train_arr:
        # Separate the label and attributes
```

```
        train_label = train_sample[-1]
        train_attributes = train_sample[:-1]

        # Calculate the Euclidean distance between the test and train attributes
        dist = np.linalg.norm(test_attributes - train_attributes)

        # Store the distance and training label
        dist_info = (dist, train_label)

        # Append the distance and training label info to the distances list
        distances.append(dist_info)

    # Return the list of distances and training labels
    return distances
```

4.2 Functions for kNN classification

```
# Split the data into training and testing using a given training percentage
(default: 70%) and seed (default: 36)
def split_data(df, train_percentage=0.7, seed=36):
    # Randomly sample the training df
    train_df = df.sample(frac=train_percentage, random_state=seed)

    # Drop the training df indices to create the testing df
    test_df = df.drop(train_df.index)

    # Return the train and test dfs
    return train_df, test_df
```

```
# Finds the k minimum samples from the given list of distance info
def find_min_samples(k, dist_info):
    # Turn the list into a df
    dist_info_df = pd.DataFrame(dist_info)

    # Sort the values into a list by their distances
    sorted_dist_info = dist_info_df.sort_values(by=0).values.tolist()

    # Get the first k (lowest distance) samples
    min_samples = sorted_dist_info[:k]
```

```
    # Return the minimum samples
    return min_samples
```

```
# Determine the classification of a sample given the minimum distance info from
training samples
def classify_sample(min_dist_info):
    positive_count = 0
    negative_count = 0

    # Increment the positive and negative count based on the minimum training
sample classifications
    for info in min_dist_info:
        if info[1] > 0:
            positive_count += 1
        else:
            negative_count += 1

    # Return 1 if more of the closest training samples were positive, 0 otherwise
    if positive_count > negative_count:
        return 1
    else:
        return 0
```

```
# Assign the samples to a class
def knn_classification(df, k):
    results = []

    # Set the initial true positive, true negative, false negative, and false
positive counts to 0
    tp_count = 0
    tn_count = 0
    fn_count = 0
    fp_count = 0

    # Split the data for training and testing
    train_df, test_df = split_data(df)

    # Convert the dataframes to numpy arrays
    train_arr = train_df.to_numpy()
```

```
    test_arr = test_df.to_numpy()

    # Iterate through each test sample in the test array
    for test_sample in test_arr:
        # Get the actual class of the test sample
        actual_class = test_sample[-1]

        # Calculate the euclidean distances between the test sample and each
sample in the training data
        dist_info = calculate_distances(train_arr, test_sample)

        # Find the k closest training samples to the test sample
        min_samples = find_min_samples(k, dist_info)

        # Predict the class of the sample based on the closest training samples
        pred_class = classify_sample(min_samples)

        # Increment the count metric
        if actual_class > 0:
            if pred_class > 0:
                tp_count += 1
            else:
                fn_count += 1
        else:
            if pred_class > 0:
                fp_count += 1
            else:
                tn_count += 1

        # Append the predicted class to the results list
        results.append(pred_class)

    # Calculate the accuracy
    tot_tp_tn = tp_count + tn_count
    tot_tp_tn_fp_fn = tp_count + tn_count + fp_count + fn_count
    accuracy = tot_tp_tn / tot_tp_tn_fp_fn

    # Create the confusion matrix
    confusion_matrix = np.array([[tp_count, fn_count],
```

```
                                [fp_count, tn_count]])

    # Return the results list, accuracy, and confusion matrix
    return results, accuracy, confusion_matrix
```

4.3 Read in and display the data

```
# Read in the dataset
dataset_path = "wdbc.data.mb_Assginment2.csv"
df = pd.read_csv(dataset_path, header=None)

# Display the dataset
df
```

4.4 Function to normalize the data

```
# Normalizes the data using Min Max Scalar normalization
def normalize(df):
    # Use MinMaxScalar() from sklearn
    scaler = MinMaxScaler()
    normalized_array = scaler.fit_transform(df)

    # Turn the normalized array into a dataframe
    normalized_df = pd.DataFrame(normalized_array)

    # Return the normalized dataframe
    return normalized_df
```

4.5 Normalize and display the data

```
# Normalize and display the data
normalized_df = normalize(df)

normalized_df
```

4.6 Test using different k values

```
# Test using k=1

results_1, accuracy_1, confusion_matrix_1 = knn_classification(normalized_df, 1)
confusion_matrix_df_1 = pd.DataFrame(confusion_matrix_1, columns=['+', '-'],
index=['+', '-'])
```

```
print(results_1)
print(f"Accuracy: {accuracy_1}")

confusion_matrix_df_1
```

```
# Test using k=3

results_3, accuracy_3, confusion_matrix_3 = knn_classification(normalized_df, 3)
confusion_matrix_df_3 = pd.DataFrame(confusion_matrix_3, columns=['+', '-'],
index=['+', '-'])

print(results_3)
print(f"Accuracy: {accuracy_3}")

confusion_matrix_df_3
```

```
# Test using k=5

results_5, accuracy_5, confusion_matrix_5 = knn_classification(normalized_df, 5)
confusion_matrix_df_5 = pd.DataFrame(confusion_matrix_5, columns=['+', '-'],
index=['+', '-'])

print(results_5)
print(f"Accuracy: {accuracy_5}")

confusion_matrix_df_5
```

```
# Test using k=7

results_7, accuracy_7, confusion_matrix_7 = knn_classification(normalized_df, 7)
confusion_matrix_df_7 = pd.DataFrame(confusion_matrix_7, columns=['+', '-'],
index=['+', '-'])

print(results_7)
print(f"Accuracy: {accuracy_7}")
confusion_matrix_df_7
```

```
# Test using k=9
```

```
results_9, accuracy_9, confusion_matrix_9 = knn_classification(normalized_df, 9)
confusion_matrix_df_9 = pd.DataFrame(confusion_matrix_9, columns=['+', '-'],
index=['+', '-'])

print(results_9)
print(f"Accuracy: {accuracy_9}")

confusion_matrix_df_9
```

Acknowledgement:

I would like to thank **Dr Mahmut Karakaya** for providing the template for the project report.